



Shared-exclusion redundancy as fault tolerance

Multisensor consistency, forecasts and injected faults

Sepehr Mahmoudian

26 September 2026

Cite this report as: Sepehr Mahmoudian (2026). *Shared-exclusion redundancy as fault tolerance for multisensor systems*. pid-rs repository research note. Include the exact repository commit that you used. The software citation is in CITATION.cff. Cite the defining method papers separately; the References section lists them.

Summary

Several sensors observe one situation, and some of them can fail. A failed sensor can report any value. This note shows that the events of the Makkeh–Gutknecht–Wibral (MGW) shared-exclusions decomposition describe exactly which true states are consistent with the reports under a stated fault assumption. It then tests the forecast that this reading defines on recorded office sensors with injected faults.

The results are of four kinds.

1. **Formal and exact results.** For an antichain α and a report r , the MGW source event around r is the set of source states whose disagreement with r is a fault set that α tolerates (Theorem 1). For the threshold antichain of all $(m - f)$ -subsets of m sources, the event is the Hamming ball of radius f (Corollary 1). The forecast $q_\alpha(t \mid r)$, the target law inside that event, gives the true target at least the true joint probability whenever the actual faults are tolerated (Theorem 2). Lean 4 checks these three results and the counts of the parity example below, using only the standard axioms.
2. **Identities.** The averaged MGW cumulative term of α equals the log-score gain of q_α over the target prior in the clean law. For the threshold antichain it is the fault-tolerant value FT_f . The loss $I(S; T) - FT_f$ equals the sum of the MGW atoms outside the down-set of that antichain. The first identity is a direct rewriting of the MGW definition; the repository states it for two sources in its office-sensor report.
3. **Limits.** FT_f need not decrease as f grows, and it can be negative. For three independent bits and their parity, the profile is $(\log 2, -\log 2, \log \frac{8}{7})$. The validity bound of Theorem 2 is weak for log loss.
4. **Experiment (development evidence).** Four physical office sensors were quantized to four levels, and one or two of them were faulted in every row. When the forecast was told the correct number of faults, its point estimates beat five fault-free forecasts in 39 of 40 random and stuck comparisons; with 12-hour bootstrap blocks, 8 of the 40 intervals include zero, all under random faults. It stayed within 0.08 nats of a Bayes forecast that knows the fault model. Told too few faults, it lost to the constant prior on one recording. Under worst-case faults





chosen for each row, it was worse than the constant prior in every condition. The last two are negative results.

The note claims no new PID measure and no scientific novelty for the forecast, which pools the training states within a Hamming ball. MGW's own operational interpretation already describes the case of the bottom node, where at least one sensor statement is true, and Milzman (2024) relates antichains to erased sources; Section 11 states what this note adds. The reading also connects MGW redundancy with the threshold and survivor-set fault models of distributed computing. All experiments use recordings that the repository had examined before; they are development comparisons, not confirmation.

1. Setting and fault model

All logarithms are natural logarithms. Information is in nats.

- There are m sources $S_1, \dots, S_m$ and one target T . Each takes values in a finite alphabet. A source state is $s = (s_1, \dots, s_m)$, and $\mathcal{S}$ is the set of all source states. A complete realization is $z = (s, t)$, and p is a probability law on these realizations.
- For a nonempty set $a \subseteq \{1, \dots, m\}$, s_a is the restriction of s to the sources in a .
- An **antichain** α is a nonempty family of nonempty source sets such that no member contains another member. The **redundancy order** is $\alpha \preceq \beta$ if every $b \in \beta$ contains some $a \in \alpha$.

Fault model. The true source state is s' . The sensors report $r \in \mathcal{S}$. A **fault set** $F \subseteq \{1, \dots, m\}$ contains the sensors that may be wrong: $r_i = s'_i$ for every $i \notin F$, and r_i is arbitrary for $i \in F$. The set of sensors that actually disagree is

$$D(r, s') = \{i : r_i \neq s'_i\}.$$

It is contained in every fault set that describes the reports. The target is not observed through the sensors, so faults change r but not t .

Tolerance. An antichain α **tolerates** a fault set F if some collection $a \in \alpha$ avoids it: $a \cap F = \emptyset$. Read each collection as a group of sensors that is assumed to be correct together. Then α tolerates F exactly when at least one such group contains no faulty sensor.

MGW source event. For an antichain α and a source state r ,

$$\mathfrak{a}_\alpha(r) = \{s \in \mathcal{S} : s_a = r_a \text{ for at least one } a \in \alpha\}.$$

This is the source part of the MGW shared-exclusion event. It is an OR across the collections of α and an AND inside each collection. MGW use base-2 logarithms; this note uses natural logarithms, which scales every information value by $\ln 2$.





2. The MGW event is a fault-consistency set

Theorem 1. For every antichain α and all source states r and s ,

$$s \in \mathbf{a}_\alpha(r) \iff \alpha \text{ tolerates } D(r, s).$$

Proof. ($\Rightarrow$) Suppose $s_a = r_a$ for some $a \in \alpha$. Every $i \in a$ has $s_i = r_i$, so $i \notin D(r, s)$. Hence $a \cap D(r, s) = \emptyset$, and α tolerates $D(r, s)$.

($\Leftarrow$) Suppose $a \in \alpha$ and $a \cap D(r, s) = \emptyset$. Then no $i \in a$ lies in $D(r, s)$, so $s_i = r_i$ for every $i \in a$. That is $s_a = r_a$, so $s \in \mathbf{a}_\alpha(r)$. $\square$

Because $D(r, s) = D(s, r)$, the statement is symmetric: the truth lies in the event around the report exactly when the report lies in the event around the truth. The tolerated fault sets are closed under subsets: if a collection avoids F , it avoids every subset of F . A true state s can produce the reports r under some tolerated fault set exactly when $D(r, s)$ is contained in one, which by that closure means that $D(r, s)$ is itself tolerated. So, given the reports r , the event $\mathbf{a}_\alpha(r)$ is the set of every true state that the reports allow if the faults are of a kind that α tolerates.

Corollary 1 (threshold antichains). Let $0 \leq f < m$ and let α_f be the antichain of all sets of $m - f$ sources. Then α_f tolerates F exactly when $|F| \leq f$, and

$$\mathbf{a}_{\alpha_f}(r) = \{s : |D(r, s)| \leq f\},$$

the Hamming ball of radius f around r .

Proof. If $|F| \leq f$, the complement of F has at least $m - f$ elements, so it contains a set a of exactly $m - f$ elements. That set avoids F and belongs to α_f . Conversely, if some $a \in \alpha_f$ avoids F , then F lies in the complement of a , which has f elements, so $|F| \leq f$. The event form follows from Theorem 1. $\square$

Two special cases fix the ends of the range. For $f = 0$ the antichain is $\{\{1, \dots, m\}\}$ and the event is the single state r : no fault is tolerated. For $f = m - 1$ the antichain is $\{\{1\}, \dots, \{m\}\}$ and the event is the OR event: at least one sensor is correct. The repository's office-sensor report tested this OR event as a forecast for two sensors.

Lemma 2 (order). If $\alpha \preceq \beta$, then $\mathbf{a}_\beta(r) \subseteq \mathbf{a}_\alpha(r)$ for every r , and α tolerates every fault set that β tolerates.

Proof. Let $s \in \mathbf{a}_\beta(r)$, so $s_b = r_b$ for some $b \in \beta$. By the order, some $a \in \alpha$ satisfies $a \subseteq b$, and then $s_a = r_a$. So $s \in \mathbf{a}_\alpha(r)$. For tolerance, if $b \cap F = \emptyset$ and $a \subseteq b$, then $a \cap F = \emptyset$. $\square$

A node lower in the redundancy order therefore tolerates at least the same fault patterns and has an event at least as large. The threshold antichains form a chain, $\alpha_0 \succeq \alpha_1 \succeq \dots \succeq \alpha_{m-1}$.





3. The consistency forecast and its information value

Definition. For a report r with $p(S \in \mathfrak{a}_\alpha(r)) > 0$, the **consistency forecast** is

$$q_\alpha(t \mid r) = p(T = t \mid S \in \mathfrak{a}_\alpha(r)).$$

For α_f it is the target law among the states within Hamming distance f of the report. Write q_f for it.

Proposition 3 (log-score identity). Let $i_\alpha^{\text{sx}}(z)$ be the MGW net pointwise cumulative term of α at a supported realization $z = (s, t)$. Then

$$i_\alpha^{\text{sx}}(z) = \log q_\alpha(t \mid s) - \log p(t), \quad \sum_z p(z) i_\alpha^{\text{sx}}(z) = H(T) - \mathbb{E}_p[-\log q_\alpha(T \mid S)].$$

Proof. MGW define the net term as $\log \frac{p(\mathfrak{t} \cap \mathfrak{a})}{p(\mathfrak{t})p(\mathfrak{a})}$, where $\mathfrak{a}$ is the event $S \in \mathfrak{a}_\alpha(s)$ and $\mathfrak{t}$ is the event $T = t$. Both events contain z , so all three probabilities are positive. The quotient $p(\mathfrak{t} \cap \mathfrak{a})/p(\mathfrak{a})$ is $q_\alpha(t \mid s)$, and $p(\mathfrak{t}) = p(t)$. Taking the expectation under p gives $\mathbb{E}[\log q_\alpha(T \mid S)] - \mathbb{E}[\log p(T)]$, and $-\mathbb{E}[\log p(T)] = H(T)$. $\square$

The identity uses the same law p for the events and the expectation, and the report equals the true state. It describes clean data. It says nothing yet about faults or about a different recording.

Definition. The **fault-tolerant value** of budget f is the averaged cumulative of the threshold antichain,

$$\text{FT}_f = H(T) - \mathbb{E}_p[-\log q_f(T \mid S)].$$

It is the log-score gain, over the target prior, of the forecast that assumes at most f faulty sensors, measured on clean data. $\text{FT}_0 = I(S_1, \dots, S_m; T)$, because the ball of radius zero is the observed state itself.

Proposition 4 (cost of tolerance). Let Π_β be the averaged MGW net atoms. Then

$$I(S_1, \dots, S_m; T) - \text{FT}_f = \sum_{\beta \not\leq \alpha_f} \Pi_\beta.$$

Proof. Möbius inversion gives, for every node γ , averaged cumulative $= \sum_{\beta \leq \gamma} \Pi_\beta$. The top node $\{\{1, \dots, m\}\}$ lies above every node, so its cumulative is the sum of all atoms, and that cumulative is $I(S_1, \dots, S_m; T)$. The cumulative of α_f is FT_f . Subtract. $\square$

For two sources and $f = 1$, $\alpha_1 = \{\{1\}, \{2\}\}$ is the redundancy node. Then FT_1 is the MGW redundancy, and the cost is the sum of the two unique atoms and the synergy atom. Tolerating one faulty sensor out of two keeps exactly the redundant part.

The informative part is monotone; the net value is not. The informative component $\mathbb{E}_p[-\log p(\mathfrak{a}_{\alpha_f}(S))]$ never increases with f , because the balls are nested (Lemma 2). The net value FT_f subtracts a misinformative component, and Section 5 shows that it can rise, fall and become negative.





4. Validity under tolerated faults

Theorem 2. Let the true realization be (s', t) with $p(s', t) > 0$, and let the report r satisfy: α tolerates $D(r, s')$. Then $p(S \in \mathfrak{a}_\alpha(r)) > 0$ and

$$q_\alpha(t | r) \geq p(s', t), \quad \text{so} \quad -\log q_\alpha(t | r) \leq -\log p(s', t).$$

Proof. By Theorem 1, $s' \in \mathfrak{a}_\alpha(r)$. The numerator of $q_\alpha(t | r)$ is $p(S \in \mathfrak{a}_\alpha(r), T = t)$; it is a sum of nonnegative masses that includes $p(s', t)$, so it is at least $p(s', t) > 0$. The denominator $p(S \in \mathfrak{a}_\alpha(r))$ includes the same mass, so it is positive, and it is at most one. A quotient of a number at least $p(s', t)$ by a positive number at most one is at least $p(s', t)$. $\square$

The bound holds for every report inside the tolerated fault patterns, including a report chosen by an adversary who knows the truth. It guarantees that the forecast never gives zero probability to the true target under tolerated faults. It does not make the forecast good: the bound is the joint probability of the whole realization, and its average, $\mathbb{E}_p[-\log p(S, T)] = H(S, T)$, is at least $H(T)$, the loss of the prior. Lean checks the probability inequality; the logarithmic form follows because the logarithm is increasing. Section 7.3 compares the size of the bound with losses on recorded data.

A standard way to cap the worst case is to mix with a prior π with $\pi(t) > 0$ for every t . Define $q_\alpha(\cdot | r)$ as any law when the event has probability zero. For $0 < \lambda < 1$, the forecast $\lambda q_\alpha + (1 - \lambda)\pi$ has log loss at most $-\log(1 - \lambda) - \log \pi(t)$ under any faults, because it is at least $(1 - \lambda)\pi(t)$. This remark is not tested here.

5. Examples: the profile need not decrease

Three-bit parity. Let S_1, S_2, S_3 be independent uniform bits and $T = S_1 \oplus S_2 \oplus S_3$. Each of the eight source states has mass $1/8$, and $p(t) = 1/2$ for each target value.

- $f = 0$: the ball is the state itself, which fixes the parity. The ratio $q_0(t | s)/p(t)$ is $1/(1/2) = 2$, so $\text{FT}_0 = \log 2$.
- $f = 1$: the ball holds s and its three neighbours. Each neighbour has the opposite parity, so one of the four states has the right target: $q_1(t | s) = 1/4$ and $\text{FT}_1 = \log \frac{1}{2}$.
- $f = 2$: the ball holds seven states; only the complement of s is missing. The states at distance 0 and 2 have the right parity, which is $1 + 3 = 4$ states: $q_2(t | s) = 4/7$ and $\text{FT}_2 = \log \frac{4/7}{1/2} = \log \frac{8}{7}$.

So the profile is $(\log 2, -\log 2, \log \frac{8}{7}) \approx (0.693, -0.693, 0.134)$. With one tolerated fault, every alternative state that the fault allows has the wrong parity, so the forecast is worse than the prior. Allowing two faults adds three states with the right parity and restores a little information. Lean checks the three counts behind these values (4 and 1 for radius one, 7 and 4 for radius two, and 4 states of each parity).

Copied target with a duplicated noise pair. Let S_1 be a uniform bit, let $T = S_1$, and let $S_2 = S_3$ be an independent uniform bit. The four source states (a, b, b) have mass $1/4$ each.

- $f = 0$: $\text{FT}_0 = I(S; T) = \log 2$.
- $f = 1$: among the supported states, the ball around (a, b, b) holds (a, b, b) and $(1 - a, b, b)$. Their targets differ, so $q_1(t | s) = 1/2$ and $\text{FT}_1 = 0$.
- $f = 2$: the ball also holds $(a, 1 - b, 1 - b)$ but not $(1 - a, 1 - b, 1 - b)$, which is at distance three. Two of the three states have target a , so $q_2(t | s) = 2/3$ and $\text{FT}_2 = \log \frac{4}{3}$.





Here more tolerated faults give more information, because the larger ball admits the state $(a, 1 - b, 1 - b)$, which has the right target. The state that disagrees on all three sensors is outside both balls.

Two-bit XOR. For $m = 2$, FT_1 is the MGW redundancy, which is $\log \frac{2}{3} < 0$ for $T = S_1 \oplus S_2$ with independent uniform bits.

6. Computation and checks

Algorithm. For a law with R supported realizations, one value FT_f needs, for each realization, the Hamming distances to all R realizations: $O(R^2m)$ operations and $O(Rm)$ memory for the table itself. No lattice is enumerated. The full antichain lattice grows quickly with m : it has 4, 18, 166, 7,579 and 7,828,352 nodes for $m = 2, \dots, 6$ (the Dedekind numbers minus two). `pid-rs` computes the full categorical decomposition for up to four sources.

Checks. All checks are retained with this note and are execution evidence on the stated inputs.

Check	Inputs	Result
pid-rs atoms summed over the down-set of α_f versus the Hamming computation	60 seeded systems, 2–4 sources, every f	Largest difference 5.0×10^{-16} nats
Same identity with a dense zeta solve; $I(S; T)$ computed directly versus the top-node cumulative; Proposition 4 with atoms from the same solve	300 random laws, 2–4 sources	Largest differences 1.1×10^{-15} , 6.7×10^{-16} and 1.8×10^{-15} nats
Theorem 2 in exact rational arithmetic	Every supported truth and every tolerated report of the 300 laws	613,270 checks; no violation
Informative component never increases with f	The 300 laws	No increase
Exact example profiles	Parity and duplicated-noise examples	As in Section 5

The first check reads the categorical dump retained with the [cross-implementation re-verification report](#), which holds the rows and the atoms that `pid-rs discrete_sxpid_n` returned. It therefore connects Corollary 1 to the Rust implementation.

7. Experiment: recorded office sensors with injected faults

7.1 Data, encoding and forecasts

The data are the three UCI Occupancy Detection recordings described in the [office-sensor report](#): 8,143 training rows, an earlier recording of 2,665 rows and a later recording of 9,752 rows, sampled about once per minute. The primary analysis uses the four physical sensors: temperature, relative humidity, light and CO_2 . The provider derives the fifth channel, humidity ratio, from temperature and humidity. On the training levels its entropy is 1.144 nats, and only 0.170 nats remain once the temperature and humidity levels are known. Independent faults on that channel are therefore not physically consistent, so the five-channel analysis is kept only as a secondary analysis.

Each sensor is cut into four equal-width levels between its training minimum and maximum; later values outside that range are clamped. Clamping affected 248 temperature values and 1 light value in the earlier recording, and 564 temperature, 27 humidity, 3 CO_2 and 1 light values in the later one. The target is the binary occupancy label. The occupied fraction is 21.2% in training, 36.5% in the





earlier recording and 21.0% in the later one. With four sensors, 8.3% of the earlier rows and 34.7% of the later rows have a clean state that never occurs in training.

Every forecast is fitted on the training rows only. Unless stated, it adds one count per label after selecting its rows. The forecasts for a report r and an assumed fault budget f are:

- **Consistency:** training rows within Hamming distance f of r (q_f above).
- **Oblivious:** training rows equal to r ; it ignores faults. For an unseen state it returns $(\frac{1}{2}, \frac{1}{2})$.
- **Oblivious with back-off:** the same, but the prior for an unseen state.
- **Subset ensemble:** the average, over all sets a of $m - f$ sensors, of the forecast from rows that equal r on a .
- **Single average:** the average over sensors i of the forecast from rows that equal r on sensor i .
- **Oracle:** the Bayes forecast under the injected fault model, which only this forecast knows. With one added count per label on likelihood-weighted counts, the smoothing is worth between 4 and 90 training rows, depending on the fault model. A second version adds instead the likelihood-weighted mass of one average training row.
- **Prior:** the training label frequencies.

Faults are injected into the evaluation recordings only, on k sensors of every row:

- **random:** each chosen sensor reports a different level, chosen uniformly;
- **stuck:** each chosen sensor reports the top level;
- **worst case:** for each row and each forecast separately, the report among all reports that differ from the truth on at most k sensors that maximizes that forecast's loss.

The main conditions give the forecasts the correct budget, $f = k$. Section 7.4 also tests $f \neq k$ and faults on only a fraction of rows. Random and stuck losses are averaged over 20 injection seeds. The score is the mean log loss in nats per row; lower is better. Intervals are 95% percentile intervals from a circular moving-block bootstrap over rows with 2,000 resamples. The tables show blocks of 720 rows (about 12 hours); blocks of 60 and 360 rows are in the results file. The rows form a time series with daily cycles, and the earlier recording spans less than two days, so even long blocks describe only variation within these recordings. They are not population confidence intervals.

7.2 Results with the four physical sensors and the correct budget

On the training law, the profile is $FT_0, \dots, FT_3 = 0.479, 0.415, 0.353, 0.181$ nats. These are clean, in-sample values of the consistency forecast: FT_1 is 87% of FT_0 . They do not say how much information survives an actual fault.

Mean log loss (nats per row):

Recording, faults, budget	Consistency	Oblivious	Oblivious with back-off	Subset ensemble	Single average	Prior
Earlier, none, $f = 1$	0.169	0.140	0.202	0.136	0.326	0.717
Earlier, random, $f = k = 1$	0.255	0.571	0.629	0.373	0.439	0.717
Earlier, stuck, $f = k = 1$	0.186	0.462	0.528	0.333	0.416	0.717
Earlier, random, $f = k = 2$	0.550	0.769	0.808	0.535	0.564	0.717
Earlier, stuck, $f = k = 2$	0.278	0.657	0.648	0.429	0.506	0.717
Earlier, worst case, $f = k = 1$	1.040	2.451	2.545	1.460	0.761	0.717
Earlier, worst case, $f = k = 2$	2.026	3.944	3.955	1.753	1.234	0.717
Later, none, $f = 1$	0.083	0.281	0.209	0.156	0.279	0.514
Later, random, $f = k = 1$	0.332	0.541	0.444	0.390	0.387	0.514
Later, stuck, $f = k = 1$	0.192	0.536	0.376	0.369	0.383	0.514
Later, random, $f = k = 2$	0.480	0.765	0.647	0.533	0.506	0.514





Recording, faults, budget	Consistency	Oblivious	Oblivious with back-off	Subset ensemble	Single average	Prior
Later, stuck, $f = k = 2$	0.331	0.679	0.495	0.476	0.497	0.514
Later, worst case, $f = k = 1$	1.843	1.940	1.874	1.502	0.672	0.514
Later, worst case, $f = k = 2$	2.203	4.108	4.108	1.905	1.138	0.514

Differences in mean log loss, consistency minus the named forecast, with 95% intervals for blocks of 720 rows (negative favours the consistency forecast):

Recording, faults, budget	vs oblivious with back-off	vs subset ensemble	vs single average	vs prior
Earlier, none, $f = 1$	-0.033 [-0.199, +0.087]	+0.033 [-0.003, +0.073]	-0.157 [-0.214, -0.100]	-0.547 [-0.792, -0.303]
Earlier, random, $f = k = 1$	-0.374 [-0.496, -0.257]	-0.118 [-0.170, -0.064]	-0.184 [-0.226, -0.145]	-0.462 [-0.702, -0.241]
Earlier, stuck, $f = k = 1$	-0.342 [-0.537, -0.161]	-0.147 [-0.194, -0.101]	-0.230 [-0.253, -0.207]	-0.531 [-0.796, -0.292]
Earlier, random, $f = k = 2$	-0.258 [-0.288, -0.220]	+0.015 [-0.147, +0.185]	-0.014 [-0.124, +0.099]	-0.167 [-0.268, -0.070]
Earlier, stuck, $f = k = 2$	-0.370 [-0.568, -0.179]	-0.151 [-0.232, -0.077]	-0.228 [-0.272, -0.188]	-0.439 [-0.687, -0.199]
Earlier, worst case, $f = k = 1$	-1.505 [-1.797, -1.195]	-0.420 [-0.632, -0.206]	+0.278 [+0.112, +0.416]	+0.323 [+0.162, +0.473]
Earlier, worst case, $f = k = 2$	-1.929 [-2.653, -1.051]	+0.273 [-0.140, +0.735]	+0.793 [+0.375, +1.238]	+1.310 [+0.938, +1.684]
Later, none, $f = 1$	-0.125 [-0.206, -0.057]	-0.073 [-0.126, -0.023]	-0.196 [-0.240, -0.157]	-0.431 [-0.582, -0.311]
Later, random, $f = k = 1$	-0.112 [-0.229, -0.009]	-0.059 [-0.122, +0.018]	-0.055 [-0.119, +0.007]	-0.183 [-0.345, -0.041]
Later, stuck, $f = k = 1$	-0.185 [-0.286, -0.091]	-0.177 [-0.217, -0.132]	-0.191 [-0.229, -0.151]	-0.323 [-0.491, -0.176]
Later, random, $f = k = 2$	-0.167 [-0.229, -0.101]	-0.052 [-0.124, +0.028]	-0.026 [-0.078, +0.028]	-0.034 [-0.126, +0.047]
Later, stuck, $f = k = 2$	-0.164 [-0.292, -0.051]	-0.145 [-0.195, -0.089]	-0.166 [-0.213, -0.117]	-0.183 [-0.353, -0.027]
Later, worst case, $f = k = 1$	-0.031 [-0.649, +0.602]	+0.341 [-0.063, +0.771]	+1.170 [+0.755, +1.597]	+1.328 [+0.853, +1.812]
Later, worst case, $f = k = 2$	-1.904 [-2.265, -1.546]	+0.298 [+0.040, +0.585]	+1.066 [+0.803, +1.328]	+1.689 [+1.397, +1.982]

Against the oracle, with its two smoothings:

Recording, faults, budget	Oracle, one added count	Oracle, one-row pseudo-count	vs oracle, one added count	vs oracle, one-row pseudo-count
Earlier, random, $f = k = 1$	0.258	0.278	-0.004 [-0.007, -0.000]	-0.024 [-0.060, -0.000]
Earlier, stuck, $f = k = 1$	0.176	0.196	+0.010 [+0.001, +0.020]	-0.010 [-0.030, +0.011]
Earlier, random, $f = k = 2$	0.539	0.552	+0.011 [+0.000, +0.023]	-0.002 [-0.012, +0.006]
Earlier, stuck, $f = k = 2$	0.251	0.260	+0.027 [+0.009, +0.046]	+0.018 [-0.003, +0.039]
Later, random, $f = k = 1$	0.314	0.346	+0.018 [+0.004, +0.036]	-0.015 [-0.043, +0.009]
Later, stuck, $f = k = 1$	0.199	0.161	-0.008 [-0.037, +0.022]	+0.030 [+0.003, +0.056]
Later, random, $f = k = 2$	0.459	0.486	+0.022 [+0.006, +0.037]	-0.006 [-0.015, +0.003]
Later, stuck, $f = k = 2$	0.251	0.280	+0.080 [+0.032, +0.132]	+0.051 [-0.033, +0.131]





7.3 What the results show

1. **Random and stuck faults with the correct budget.** Over the 40 comparisons with the five fault-free forecasts (two recordings, two budgets, two fault kinds), the point estimate favoured the consistency forecast in 39. The exception is the subset ensemble for random faults with $f = k = 2$ on the earlier recording (+0.015). The intervals cross zero in 4 comparisons with blocks of 60 rows and in 8 with blocks of 360 or 720 rows. Every crossing is under random faults: seven involve the subset ensemble, the single average or the prior, and one involves the plain oblivious forecast. Stuck faults give no crossing.
2. **The oracle.** The consistency forecast was within 0.08 nats of both oracle versions in all 8 conditions, sometimes better. The oracle is a smoothed training-law Bayes forecast, not the true Bayes risk, and its result depends on the smoothing; it is not a ceiling.
3. **No faults.** On the earlier recording, the differences from the oblivious forecasts and the subset ensemble have intervals that include zero; the single average and the prior are worse. On the later recording, the consistency forecast was better than every other forecast. That recording has the most states unseen in training (34.7%), so the pooling of nearby states acts as smoothing.
4. **Worst-case faults.** The consistency forecast was worse than the constant prior in all four conditions, by 0.32 to 1.69 nats, and worse than the single-sensor average. An adversary who knows the truth can choose a report whose ball is dominated by the other label.
5. **Theorem 2 on these data.** Theorem 2 concerns the law used to fit the forecast. It applies only to rows whose true state and label occur together in training: 91.7% of the earlier rows and 65.1% of the later rows. On those rows the mean of the bound $-\log \hat{p}(s', t)$ is 3.46 and 3.81 nats, while the mean worst-case loss of the smoothed forecast is about 1.0 for $f = 1$ and 1.7 to 1.8 for $f = 2$. On the other rows the mean worst-case losses are 1.7 to 4.05 nats. The scored forecast is smoothed and fitted on another recording, so it lies outside the theorem's premises: smoothing alone can move it below the bound, for example to $61/102 < 0.6$ when 60 of 100 rows equal (s', t) and the ball holds all rows. These numbers show the scale of the bound; they do not test it.

7.4 Wrong budgets and partial fault rates

The table gives the consistency forecast when the assumed budget f differs from the injected count k , with 720-row intervals:

Recording, faults, assumed f , injected k	Consistency	vs subset ensemble	vs single average	vs prior
Earlier, random, $f = 1$, $k = 2$	0.685	+0.075 [−0.062, +0.207]	+0.121 [+0.063, +0.178]	−0.032 [−0.210, +0.151]
Earlier, stuck, $f = 1$, $k = 2$	0.385	−0.124 [−0.196, −0.049]	−0.122 [−0.182, −0.056]	−0.332 [−0.630, −0.049]
Earlier, random, $f = 2$, $k = 1$	0.352	+0.010 [−0.072, +0.096]	−0.088 [−0.123, −0.055]	−0.365 [−0.545, −0.205]
Earlier, stuck, $f = 2$, $k = 1$	0.234	−0.056 [−0.108, −0.005]	−0.181 [−0.221, −0.144]	−0.483 [−0.743, −0.248]
Later, random, $f = 1$, $k = 2$	0.730	+0.099 [+0.045, +0.157]	+0.225 [+0.176, +0.275]	+0.216 [+0.092, +0.331]
Later, stuck, $f = 1$, $k = 2$	0.459	−0.096 [−0.132, −0.058]	−0.037 [−0.094, +0.021]	−0.055 [−0.246, +0.116]
Later, random, $f = 2$, $k = 1$	0.253	−0.084 [−0.155, −0.004]	−0.134 [−0.175, −0.077]	−0.261 [−0.370, −0.176]
Later, stuck, $f = 2$, $k = 1$	0.177	−0.130 [−0.189, −0.060]	−0.207 [−0.245, −0.159]	−0.337 [−0.473, −0.232]





Too small a budget ($f = 1, k = 2$) with random faults removes the advantage: on the later recording the consistency forecast is worse than all three forecasts shown, including the constant prior (+0.216). With stuck faults it keeps an advantage over the subset ensemble. Too large a budget ($f = 2, k = 1$) costs little and keeps most comparisons favourable.

When only a random fraction ρ of rows is faulted, the expected loss mixes the clean and the faulted losses linearly. On the earlier recording, with $f = 1$, the consistency forecast beats the subset ensemble only for $\rho > 0.22$, and the plain oblivious forecast only for $\rho > 0.085$; with $f = 2$ the break-even points against the oblivious forecasts are 0.26 (plain) and 0.049 (with back-off), and it never beats the subset ensemble. On the later recording it is better at every fault rate, because it is already better on clean data.

7.5 The secondary five-channel analysis

With humidity ratio added as a fifth channel, the pattern is the same. With the correct budget, the point estimates favour the consistency forecast against the four fault-free forecasts in random and stuck conditions, and it loses to the prior under worst-case faults. The profile is 0.481, 0.404, 0.314, 0.260, 0.159 nats. All numbers are in the results file.

7.6 Choosing three sensors

For each of the ten sets of three of the five channels, the table gives four training-law criteria and three losses on the later recording with a budget of one fault. “Worst single erasure” is the smallest mutual information left after removing one of the three channels. T, H, L, C and W denote temperature, humidity, light, CO₂ and humidity ratio.

Sensors	MI	FT ₁	FT ₂	Worst single erasure	MI minus FT ₁	Oblivious, random	Consistency, random	Consistency, worst case
THL	0.476	0.412	0.243	0.204	0.064	0.562	0.425	1.999
THC	0.297	0.221	0.120	0.204	0.076	0.747	0.530	1.555
THW	0.209	0.074	0.080	0.051	0.135	0.818	0.469	1.084
TLC	0.475	0.406	0.266	0.258	0.069	0.592	0.298	1.178
TLW	0.475	0.406	0.236	0.199	0.070	0.581	0.389	1.840
TCW	0.282	0.224	0.119	0.199	0.058	0.733	0.520	1.557
HLC	0.475	0.385	0.219	0.251	0.090	0.635	0.407	1.561
HLW	0.471	0.166	0.264	0.051	0.305	0.854	0.316	1.098
HCW	0.259	0.077	0.088	0.051	0.182	0.854	0.549	1.206
LCW	0.473	0.370	0.217	0.236	0.104	0.698	0.347	1.301

The set HLW has almost the largest mutual information (0.471), but the largest loss of clean information under one tolerated fault ($MI - FT_1 = 0.305$): without light, humidity and humidity ratio together carry only 0.051 nats about occupancy. The worst single erasure flags the same weakness, and FT_2 does not. The three sets with the largest loss all contain the humidity and humidity-ratio pair, which is nearly redundant. This is one post hoc example.

Over the ten sets, the Spearman rank correlations of FT_1 and of mutual information with the losses are similar. Both correlate strongly with clean consistency performance (0.94 to 0.95), and both correlate negatively with worst-case consistency performance (−0.68 and −0.70 on the later recording): more informative sets give more confident forecasts, which an adversary can exploit. Ten overlapping sets are too few to rank the criteria; every correlation is in [results/experiment-v3.json](#).





7.7 Limits of the experiment

The three recordings come from one office and had been examined before this experiment was designed, so the results are development evidence. The quantizer, the fault models, the budgets and the choice of sensors were chosen by the author, and no protocol was registered in advance. Every faulted row has exactly k faults, and the main comparisons give the forecasts the correct k . Real sensor faults include drift, delay, intermittent failure and correlated failure. The oracle and the fault models are simple. The block bootstrap does not model the shift between recordings. No claim is made for other rooms, sensors, quantizers or targets.

8. Negative results and their reasons

- **Worst-case faults.** The consistency forecast lost to the constant prior under worst-case faults. Validity (Theorem 2) is a zero-avoidance guarantee, not a minimax guarantee.
- **Too small a budget.** With two random faults and an assumed budget of one, the consistency forecast was worse than the prior, the subset ensemble and the single average on the later recording.
- **Few faulted rows.** When few rows are faulted, the clean-data cost decides: on the earlier recording the consistency forecast beats the subset ensemble only when more than 22% of rows are faulted ($f = 1$), and never for $f = 2$.
- **Interval sensitivity.** With 60-row blocks, 4 of the 40 matched-budget comparisons have intervals that include zero; with 360- or 720-row blocks, 8 do. The first version of this note reported only 60-row blocks and claimed that every interval excluded zero; that claim did not survive longer blocks.
- **Negative correlations.** Across the ten sensor sets, both mutual information and FT_1 correlate negatively with worst-case consistency performance.
- **Non-monotone and negative profiles.** Parity gives $FT_1 = -\log 2$, and the duplicated-noise example gains information as f grows. A fault-tolerant value is not a monotone “robust information” function.
- **Two-sensor OR forecast.** The office-sensor report found the OR forecast ($f = m - 1$ for $m = 2$) worse than light alone and joint matching on clean data. That is the most permissive ball, and the present results agree that large budgets cost clean performance.
- **Sensor selection.** The subset study does not show that ranking by FT_1 selects better sensor sets than mutual information or the worst single erasure.

9. What this adds for sensors and physical intelligence

- **A one-parameter fusion rule with a meaning.** Choose the number f of sensors to tolerate. The forecast pools training states within Hamming distance f of the reports. Its clean value is the MGW cumulative FT_f (Proposition 3), its loss of clean information is a sum of named MGW atoms (Proposition 4), and it never assigns zero probability to the truth under tolerated faults (Theorem 2), for true states and labels that occur in the fitting law. On these recordings, when told the correct number of faulty sensors, it was close to a Bayes forecast that knows the fault model, for random and stuck faults, without knowing which kind of fault occurred. It needs a budget at least as large as the actual fault count, and it is not robust to worst-case faults.





- **A degradation profile.** The values $f \mapsto \text{FT}_f$ summarize how much clean target information the consistency forecast keeps when it must tolerate f faults. In one post hoc example (HLW), the loss $\text{MI} - \text{FT}_1$ exposed a single point of failure that mutual information alone hides; the worst single erasure showed the same weakness.
- **Beyond thresholds.** Theorem 1 holds for every antichain, not only thresholds. An antichain is a list of sensor groups, each assumed correct together; this is the survivor-set description of dependent failures in distributed computing (Section 11). A designer can encode, for example, “the two cameras share a power supply” by the choice of groups.
- **Placement.** For sensor placement, a candidate layout can be scored by FT_f at the required budget, and then tested with task loss under a declared fault model on new recordings. The [sensor-placement guide](#) describes the required evidence gates. No placement improvement is established here.

10. Routes considered

Each route lists its assumption, the decision and what would change it.

1. **No action.** *Assumption:* existing analyses cover sensor faults. *Decision:* rejected; the repository had no fault model. *Would change if:* a fault study existed.
2. **Mutual information for sensor selection.** *Assumption:* more information means more robustness. *Decision:* kept as a baseline; it missed the HLW single point of failure.
3. **Worst single erasure (removal).** *Assumption:* faults remove sensors. *Decision:* kept as a baseline; it also flags HLW, but it models erasure, not wrong values.
4. **Oblivious forecast.** *Assumption:* no faults. *Decision:* baseline; it degrades sharply under faults.
5. **Leave- f -out subset ensemble.** *Assumption:* averaging reduces fault damage. *Decision:* baseline; it lost to the consistency forecast under stuck faults, and under random faults in most comparisons.
6. **Single-sensor average.** *Assumption:* one faulty sensor affects one term. *Decision:* baseline; it was the most robust non-constant forecast under worst-case faults.
7. **Fault-aware Bayes forecast.** *Assumption:* the fault model is known. *Decision:* used as an oracle for the injected channel. Deployment would require a justified or estimated fault model.
8. **Consistency forecast from MGW threshold events.** *Decision:* **selected** as the object of study. *Would change if:* it lost to simpler forecasts under random faults on new recordings.
9. **Minimax forecast for worst-case faults.** *Decision:* not attempted; it needs a different optimization. *Would change if:* worst-case robustness became the design goal.
10. **Mixture with the prior.** *Decision:* stated as a bound (Section 4), not tested.
11. **Continuous analogue.** The Ehrlich source-disjunction distance is the continuous version of the OR event. *Decision:* future work; its fault reading needs a separate theorem.





11. Relation to prior work

- **MGW (2021), Section V.B**, give the operational interpretation closest to this note. A channel emits a true statement $(S_1 = s_1) \vee \dots \vee (S_n = s_n)$ whose substatements may be false, and a receiver who knows the joint law infers t in a Bayes-optimal way; the measure averages only over the uses in which every substatement is true. That receiver's forecast is the consistency forecast of the bottom node $\{\{1\}, \dots, \{m\}\}$ ($f = m - 1$), and the restriction to true substatements is the clean-data caveat of Proposition 3. This note extends that reading to every antichain (Theorem 1), names the threshold antichains and their Hamming balls, and adds the validity bound, the cost identity, the examples and the fault-injection test.
- **Gutknecht, Wibral and Makkeh (2021)** describe PID atoms through part-whole relations and formal logic. In that reading an antichain is a disjunction over its collections, and each collection is a conjunction of sources. The threshold antichain α_f is then the disjunction over all groups of $m - f$ sources, which is the consistency statement used here.
- **Marzullo (1990)** fuses interval readings from sensors of which at most f are faulty by keeping the values consistent with at least $m - f$ sensors. Corollary 1 is the categorical analogue of that consistency set.
- **Junqueira, Marzullo, Herlihy and Draque Penso (2010)** replace the threshold model by survivor sets to express dependent failures: in every run, at least one survivor set contains only correct processes. The collections of an MGW antichain play that role, because α tolerates F exactly when some collection contains no faulty sensor. **Malkhi and Reiter (1998)** use general fail-prone systems for Byzantine quorums. The complements of the collections of α form such a system: the tolerated fault sets are exactly the subsets of those complements. Theorem 1 states the consistency set that these structures define for categorical reports.
- **Milzman (2024)** studies sensors that fail by erasure. A failure pattern “redundantly satisfies” an antichain when some collection is always fully available (his Definition 3); this is the erasure analogue of tolerance. His Theorem 1, an order-reversing correspondence between antichains and failure families, contains the tolerance half of Lemma 2, and he uses the threshold antichain of all $(n - \ell)$ -subsets for up to ℓ failures. His redundancy measure I_{ft} minimizes mutual information over those failure patterns, so it is monotone in ℓ and lies between 0 and $I(T; X)$. This note keeps the MGW measure, whose threshold values FT_f need not be monotone (Section 5), and treats faults that report wrong values rather than erasures.

12. Rust implementation and computational cost

pid-rs has no API for the fault-tolerant value or the consistency forecast. The retained code is research Python. `pid-rs discrete_sxpid_n` computes the full decomposition for up to four sources, from which FT_f is a down-set sum, as the cross-check confirms. A direct implementation would take the empirical table of R occupied source–target states, compute FT_f in $O(R^2m)$ time and $O(Rm)$ memory, and build the forecast for any report in $O(Rm)$ time; a precomputed table over all $\prod_i |\mathcal{S}_i|$ reports gives constant-time lookup at that memory cost. With four levels, the table has 256 entries for the four physical sensors and 1,024 for five channels. The third experiment version ran in 26 s on one core, including 20 injection seeds, both analyses, three block lengths and 2,000 bootstrap resamples per comparison. These are research timings, not benchmarks. A production API would need the repository's resource budgets, a declared fault budget, the prior-mixture option and tests against the checks of Section 6.





13. Evidence, reproduction and review

The directory holds:

- `lean/FaultTolerantInformation.lean`: Theorems 1 and 2, Corollary 1, Lemma 2 and the parity counts, with `lean/AxiomReceipt.lean` and its output `results/lean-axioms.txt`. The file compiles against the repository's pinned Lean 4.33.0 and Mathlib project in `audit/formal/lean` without adding files to it.
- `python/theory_checks.py` and `python/check_threshold_down_sets.py`, with outputs in `results/`.
- `python/fault_tolerant_experiment_v3.py` and its output `results/experiment-v3.json`, which Section 7 reports. The earlier versions are kept with their outputs. Version 1 (script, output) used one injection seed and also reported Brier scores. Version 2 (script, output) added 20 seeds and 60-row block intervals on all five channels. Version 3 answers the review of version 2: the four physical sensors as the primary analysis, three block lengths, wrong budgets, partial fault rates, a back-off baseline, two oracle smoothings, applicability of Theorem 2 and every subset correlation.
- `run.sh`, which reproduces every output, and `MANIFEST.json`. A confirmation run matched every retained output byte for byte; `results/reproduction-confirmation.txt` records it.

Checker correction. The original checker could hide JSON NaN in its error maximum and drop extra rows through `zip`. It now rejects nonfinite values, unequal columns and invalid categorical or antichain inputs before comparison. Its 26 subprocess controls pass in normal and optimized Python; the historical checker fails seven. The valid 60-system output is byte-identical. No theorem, forecast, experiment or tolerance changed.

The UCI files are not retained; `run.sh` takes their directory and checks their SHA-256 values. The review record is `REVIEW.md`. Evidence classes: formal proof (Lean, bounded to the encoded statements), execution evidence (the checks and the experiment), and model review. No human review is claimed.

References

- K. Marzullo (1990). Tolerating failures of continuous-valued sensors. *ACM Trans. Comput. Syst.* 8(4), 284–304. DOI 10.1145/128733.128735.
- D. Malkhi and M. Reiter (1998). Byzantine quorum systems. *Distributed Computing* 11(4), 203–213. DOI 10.1007/s004460050050.
- F. P. Junqueira, K. Marzullo, M. Herlihy and L. Draque Penso (2010). Threshold protocols in survivor set systems. *Distributed Computing* 23(2), 135–149. DOI 10.1007/s00446-010-0107-3.
- A. Makkeh, A. J. Gutknecht and M. Wibral (2021). Introducing a differentiable measure of pointwise shared information. *Phys. Rev. E* 103, 032149. arXiv:2002.03356v5.
- A. J. Gutknecht, M. Wibral and A. Makkeh (2021). Bits and pieces: understanding information decomposition from part-whole relationships and formal logic. *Proc. R. Soc. A* 477(2251), 20210110. DOI 10.1098/rspa.2021.0110.
- J. Milzman (2024). Measuring the redundancy of information from a source failure perspective. arXiv:2404.01470v1.
- L. M. Candanedo and V. Feldheim (2016). Accurate occupancy detection of an office room from light, temperature, humidity and CO₂ measurements using statistical learning models. *Energy and Buildings* 112, 28–39. Data: UCI Occupancy Detection.

